

Build a production-grade Autonomous Government Contracting Intelligence System inside:

~/AI\_SYSTEM/govcon\_agent

This is NOT a chatbot.

This is an operational contracting intelligence platform.

#### PRIMARY OBJECTIVE:

Create an AI-assisted federal contracting system that:

- monitors SAM.gov opportunities
- analyzes solicitations
- extracts requirements
- tracks deadlines
- drafts proposal artifacts
- performs bid/no-bid analysis
- manages compliance workflows
- assists with acquisition document intelligence

#### TECH STACK:

- Python 3.12
- FastAPI backend
- Next.js frontend dashboard
- PostgreSQL
- Redis queue
- Dockerized services
- Ollama/OpenAI compatible model routing
- MCP-based modular tool architecture
- Tailscale-ready remote access
- Linux-first deployment

#### ARCHITECTURE:

Use deterministic orchestration.

NO agent spaghetti.

NO uncontrolled autonomy.

#### SYSTEM MODULES:

##### 1. ORCHESTRATOR

Create:

- orchestrator.py
- task\_router.py
- workflow\_engine.py
- event\_bus.py

Responsibilities:

- route tasks
- maintain state
- execute workflows
- retry failed tasks
- maintain audit logs

## 2. VOICE + TRANSCRIPTION

Implement:

- Whisper transcription
- push-to-talk CLI mode
- microphone capture
- transcript cleanup
- transcript memory history

Support:

- local Whisper
- fallback cloud transcription

## 3. MCP TOOL SYSTEM

Create modular MCP-compatible tools:

tools/

- sam\_gov/
- fpds/
- pdf\_parser/
- proposal\_writer/
- email/
- calendar/
- compliance/
- pricing/
- crm/

Each tool must:

- expose schema
- support validation
- return structured JSON
- log execution history

## 4. SAM.GOV INGESTION

Implement:

- solicitation search
- NAICS filtering

- set-aside filtering
- due date tracking
- attachment downloading
- amendment monitoring

Store:

- solicitation metadata
- attachments
- historical changes

## 5. DOCUMENT INTELLIGENCE

Implement:

- PDF parsing
- OCR fallback
- requirement extraction
- CLIN extraction
- deliverable extraction
- compliance matrix generation
- key date extraction

Use:

- PyMuPDF
- OCR
- structured parsing pipeline

## 6. AI ANALYSIS ENGINE

Create:

- bid/no-bid scoring
- competition analysis
- pricing realism analysis
- risk scoring
- margin estimation
- contract fit analysis

Track:

- historical opportunities
- wins/losses
- competitor trends

## 7. MEMORY + RAG

Implement:

- local vector database
- retrieval system
- proposal history

- clause retrieval
- past performance indexing

Use:

- ChromaDB or Qdrant

## 8. DASHBOARD

Build:

- opportunity pipeline
- deadlines
- compliance status
- active proposals
- AI recommendations
- notification center
- proposal progress tracker

UI:

- dark mode
- operations center aesthetic
- responsive
- keyboard shortcut friendly

## 9. WATCHDOG SERVICES

Implement background services:

- deadline alerts
- amendment alerts
- expiring registrations
- Q&A reminders
- proposal due reminders

Support:

- desktop notifications
- terminal alerts
- email alerts later

## 10. PROPOSAL GENERATION

Generate:

- compliance matrix
- proposal outline
- staffing concepts
- technical approach drafts
- executive summaries
- cover letters

All outputs must:

- remain human review gated
- never auto-submit
- preserve audit history

## 11. SECURITY + COMPLIANCE

Implement:

- encrypted secrets
- RBAC-ready architecture
- audit logs
- secure credential storage
- environment variable management
- API throttling

## 12. DATABASE

Create schema for:

- opportunities
- agencies
- amendments
- tasks
- proposals
- deadlines
- notes
- competitors
- historical bids
- users
- notifications

## 13. FILE STRUCTURE

Create:

backend/  
frontend/  
workers/  
tools/  
memory/  
logs/  
exports/  
database/  
docs/  
tests/

## 14. TESTING

Implement:

- unit tests
- workflow tests
- parser tests
- API validation
- retry testing
- logging validation

## 15. DELIVERY PRIORITY

### PHASE 1:

- orchestrator
- SAM ingestion
- dashboard skeleton
- database
- deadline tracking

### PHASE 2:

- PDF intelligence
- proposal generation
- RAG memory

### PHASE 3:

- advanced analytics
- pricing intelligence
- CRM/email/calendar integrations

### REQUIREMENTS:

- fix all linting errors automatically
- run tests continuously
- commit clean atomic commits
- avoid placeholders
- production-grade structure only
- continue implementing until MVP is operational

DO NOT stop after generating skeleton files.  
Build working systems.